Optimizing Processor Efficiency: A Study of CPU Scheduling Strategies

Introduction

In any multitasking Operating System, the CPU scheduler acts as a digital traffic controller, deciding which process earns the right to use the processor at any given moment. This selection process is vital because it determines how smoothly a system runs. Especially in environments with diverse workloads—ranging from quick user clicks to heavy data processing—the choice of a scheduling algorithm can mean the difference between a responsive workstation and a sluggish, bottlenecked server.

Analyzing Fundamental Algorithms

The most basic approach is **First-Come, First-Served (FCFS)**. While it is simple to program, it suffers from the "Convoy Effect," where a single, massive task blocks a line of smaller tasks, dragging down the entire system's efficiency. To counter this, the **Shortest Job First (SJF)** method prioritizes tasks based on their execution length. While SJF is technically the best way to lower the average wait time, it is difficult to use in practice because the system cannot always predict exactly how long a new task will take to finish.

For modern, interactive environments, **Round Robin (RR)** is often the preferred choice. It gives every process a small "time slice" or quantum. If the task isn't done when time runs out, the CPU moves to the next one. The success of Round Robin depends entirely on the size of this time slice: if it is too small, the system wastes too much energy switching between tasks; if it is too large, the system starts to behave like the inefficient FCFS model.

Measuring System Performance

To judge if a scheduler is actually "good," we look at specific data points. The two most common are **Waiting Time** and **Turnaround Time**. Turnaround time represents the total duration from when a job enters the system until it is fully completed, calculated as:

$$Turnaround\ Time = Completion\ Time - Arrival\ Time$$

While SJF excels at reducing turnaround times for the whole system, Round Robin is far better at improving **Response Time**, ensuring that users feel the system is reacting to their inputs immediately, even if the total job takes longer to finish.

The Problem of Starvation and the "Aging" Solution

One major risk in priority-based systems is **Starvation**. This occurs when low-priority, long-running tasks are constantly pushed to the back of the line by newer, faster jobs, potentially waiting forever to be executed. To fix this, developers use **Aging**. This technique slowly bumps

up the priority of a task the longer it sits in the queue. Eventually, even the lowest-priority "old" task becomes the highest priority, ensuring that no job is left behind indefinitely.

Conclusion and Final Recommendation

For a server that needs to handle both interactive users and heavy background calculations, a **Multilevel Feedback Queue (MLFQ)** is the ideal solution. MLFQ is an adaptive system that shifts tasks between different queues based on their behavior. It rewards quick, interactive tasks with high priority while moving heavy, processor-hogging jobs to lower-priority queues. By combining the speed of SJF with the fairness of Round Robin and the safety of Aging, MLFQ provides a balanced, high-performance environment suitable for modern computing needs.